PRÉPARATION AUX QUESTIONS DU JURY
Guide de Réponses et Conseils

Soutenance de Licence 3 Informatique — Université d'Antsiranana
JESE Andriamaniraka Davida — Clinique GODIE

Conseils généraux pour répondre aux questions

Voici quelques conseils pour répondre avec assurance aux questions du jury :

- **Écoutez attentivement** la question jusqu'au bout avant de commencer à répondre. Ne coupez pas le jury.
- **Prenez un instant de réflexion** avant de répondre. Il vaut mieux une seconde de silence qu'une réponse précipitée.
- **Structurez votre réponse** : commencez par une phrase d'introduction, développez deux ou trois points clés, puis concluez brièvement.
- **Restez honnête** : si vous ne savez pas, dites-le plutôt que d'inventer. « Je n'ai pas approfondi cet aspect, mais je pense que... » est une réponse acceptable.
- **Faites des liens avec votre projet** : chaque réponse doit illustrer un aspect concret de votre réalisation.
- **Maintenez le contact visuel** avec les membres du jury pendant vos réponses.
- **Gérez votre temps** : une réponse ne devrait pas dépasser 1 à 2 minutes.

Les 20 questions les plus probables du jury

Q1. Pourquoi avoir choisi le localStorage plutôt qu'une base de données classique comme PostgreSQL ou MySQL ?

Le choix du localStorage répond à deux contraintes du projet. Premièrement, l'autonomie : l'application fonctionne sans dépendance serveur, ce qui la rend immédiatement démontrable sans infrastructure backend. Deuxièmement, la simplicité : le localStorage est une API native du navigateur, sans installation. L'architecture en couches que j'ai mise en place permet de remplacer facilement le localStorage par une API REST en production — il suffirait de modifier le hook useCollection pour qu'il effectue des requêtes HTTP au lieu d'accéder au localStorage. C'est donc un choix de démonstration, pas une limitation architecturale.

Q2. Qu'est-ce que l'architecture « config-driven » et pourquoi l'avoir choisie ?

L'architecture config-driven consiste à décrire chaque module non pas par du code, mais par une configuration déclarative — un objet JavaScript définissant les champs, les colonnes, les filtres et les options de recherche. Le composant CrudPage consomme cette configuration et génère automatiquement toute l'interface. J'ai choisi cette approche pour trois raisons : la productivité (ajouter un module prend quelques lignes), la cohérence (tous les modules partagent exactement la même interface) et la maintenabilité (une correction dans le framework se propage à tous les modules). Sans cette approche, j'aurais dû écrire 30 pages séparées avec un code largement répétitif.

Q3. Combien de modules comporte l'application et lesquels sont les plus importants ?

L'application comporte 30 modules fonctionnels regroupés en 8 catégories : Patients et Soins, Personnel, Consultations, Hospitalisation, Laboratoire, Pharmacie, Facturation, Urgences et Sécurité. Les modules les plus importants sont les Patients (point d'entrée du parcours de soins), les Consultations (dossier médical numérique), les Rendez-vous (calendrier interactif) et la Facturation (aspect financier). Mais tous les modules sont interconnectés : une consultation est liée à un patient et un médecin, une hospitalisation à un patient et un lit, une facture à un patient et éventuellement une assurance.

Q4. Comment garantissez-vous l'unicité des identifiants dans l'application ?

Chaque collection utilise un compteur d'auto-incrément persistant, stocké dans le localStorage sous la clé « collection__seq ». À chaque ajout, le compteur est incrémenté et sa valeur devient l'identifiant du nouvel enregistrement. Cela garantit des identifiants uniques, séquentiels et commençant à 1. La règle de gestion RG-01 formalise ce comportement. Lors de la réinitialisation, le compteur revient à zéro et les prochains enregistrements recommencent à 1.

Q5. Pourquoi avoir utilisé TypeScript plutôt que JavaScript pur ?

TypeScript apporte le typage statique, qui détecte de nombreuses erreurs à la compilation plutôt qu'à l'exécution. Dans un projet avec 30 modules et une trentaine de types d'entités, le typage est indispensable pour garantir la cohérence du code. Il améliore aussi l'autocomplététion dans l'éditeur, facilite le refactoring et rend le code plus lisible pour un futur développeur. Le surcoût de TypeScript est minime et largement compensé par la réduction des bugs.

Q6. Comment l'application gère-t-elle la sécurité des données médicales ?

La sécurité est assurée à plusieurs niveaux. D'abord, la validation des données : tous les formulaires valident les champs obligatoires et les types avant soumission. Ensuite, la protection XSS : React échappe automatiquement le contenu JSX, empêchant l'injection de scripts. L'application n'utilise pas dangerouslySetInnerHTML. Ensuite, la gestion des rôles : chaque utilisateur a un rôle qui détermine les modules accessibles, implémentant le principe du moindre privilège. Enfin, le journal des activités assure la traçabilité de chaque action avec horodatage et identification de l'utilisateur.

Q7. Quel modèle de conception avez-vous suivi pour la base de données ?

J'ai suivi la méthode Merise : un Modèle Conceptuel de Données (MCD) identifie les entités et leurs associations avec cardinalités, puis un Modèle Logique de Données (MLD) transforme le MCD en relations avec clés primaires et clés étrangères. Par exemple, la relation « un patient a plusieurs consultations » devient une clé étrangère id_patient dans la table Consultation. Cette approche garantit l'intégrité référentielle et la cohérence des données.

Q8. Comment l'application se comporte-t-elle quand la base de données est vide ?

L'application démarre sur une base vide, conformément à la règle RG-08 et au besoin BNF-09. Quand une collection est vide, le tableau affiche un état vide informatif avec un message invitant l'utilisateur à enregistrer des données. Le tableau de bord affiche également un état vide. Dès le premier enregistrement, l'identifiant est 1, grâce au compteur d'auto-incrément qui démarre à zéro et s'incrément à chaque ajout.

Q9. Quels diagrammes UML avez-vous réalisés et pourquoi ?

J'ai réalisé plusieurs diagrammes UML : un diagramme de cas d'utilisation général et un spécifique aux patients, qui modélisent les interactions entre acteurs et fonctionnalités ; un diagramme de classes qui décrit la structure statique avec une trentaine d'entités et leurs relations ; deux diagrammes de séquence qui détaillent les interactions dynamiques pour la prise de rendez-vous et la création de consultation ; un diagramme d'activité qui montre le flux de décisions lors d'une consultation ; et les modèles MCD et MLD pour la base de données. Ces diagrammes m'ont permis de valider la cohérence du modèle avant l'implémentation.

Q10. Comment avez-vous testé l'application ?

J'ai défini 26 cas de tests fonctionnels couvrant tous les modules et toutes les fonctionnalités : création, lecture, modification, suppression, recherche, filtres, pagination, exports PDF/Excel, sauvegarde JSON, mode sombre, responsive. Chaque test a été exécuté manuellement avec un protocole précis : préparation, exécution, observation, comparaison avec le résultat attendu. Les 26 tests ont tous passés avec succès. La matrice de traçabilité a confirmé que tous les besoins identifiés ont été couverts.

Q11. Pourquoi avoir choisi React plutôt qu'Angular ou Vue.js ?

React a été choisi pour sa maturité, son écosystème riche et sa communauté active. Son modèle de composants fonctionnels avec hooks permet une organisation claire du code et une réutilisation maximale, ce qui est essentiel pour un projet avec 30 modules. Angular est plus structuré mais plus lourd, avec une courbe d'apprentissage plus raide. Vue.js est excellent mais a un écosystème plus restreint. React offre le meilleur compromis entre flexibilité, performance et disponibilité de documentation.

Q12. Comment fonctionne le système de recherche dans l'application ?

Chaque module définit un champ « searchFields » dans sa configuration, qui liste les champs utilisés par la recherche. Quand l'utilisateur tape dans la barre de recherche, le composant CrudPage filtre instantanément les enregistrements en comparant le texte saisi avec les valeurs des champs définis dans searchFields. Par exemple, pour les patients, la recherche porte sur le nom, le prénom et le téléphone. Le filtrage est en temps réel, sans bouton de validation, ce qui rend la recherche fluide.

Q13. L'application est-elle utilisable sur mobile ?

Oui, l'application est entièrement responsive. J'ai utilisé les breakpoints de Tailwind CSS : mobile (moins de 640px), tablette (640 à 1024px), ordinateur (plus de 1024px). Sur mobile, la barre latérale se transforme en menu coulissant accessible via un bouton hamburger, les tableaux deviennent scrollables horizontalement, et les boutons s'adaptent à la largeur de l'écran. J'ai testé l'affichage sur un écran de 390px et l'interface s'adapte correctement.

Q14. Comment l'application gère-t-elle les exports PDF et Excel ?

Pour le PDF, j'utilise la bibliothèque jsPDF qui génère un document PDF côté client, sans dépendance serveur. Le PDF contient le titre du module, la date d'export et un tableau des données. Pour Excel, je génère un fichier CSV (Comma-Separated Values) avec les en-têtes de colonnes et les données, ce qui s'ouvre directement dans Excel. L'impression utilise la fonction native du navigateur avec un format optimisé. Ces trois exports sont disponibles dans chaque module via la barre d'outils.

Q15. Quelles sont les limites actuelles de l'application ?

La principale limite est l'absence de backend : le localStorage stocke les données dans un seul navigateur, ce qui ne permet pas une utilisation multi-postes simultanée. En production, il faudrait intégrer une API REST avec PostgreSQL. Deuxièmement, l'authentification n'est pas encore implémentée : les rôles et permissions sont définis dans le modèle mais pas encore appliqués à l'interface. Troisièmement, le localStorage a une limite de taille (environ 5 à 10 Mo), ce qui pourrait devenir contraignant avec un grand volume de données.

Q16. Comment l'application gère-t-elle les relations entre les données ?

Les relations sont gérées par le hook useRefStore, qui maintient un store centralisé des références entre collections. Quand un tableau affiche une colonne de type « ref », le composant CrudPage résout la clé étrangère en libellé affichable. Par exemple, dans la liste des consultations, la colonne « Patient » affiche le nom du patient à partir de son identifiant, et la colonne « Médecin » affiche le nom du médecin. Cette résolution est automatique et transparente pour l'utilisateur.

Q17. Quel est l'intérêt du journal des activités ?

Le journal des activités assure la traçabilité de chaque action effectuée dans le système : création, modification ou suppression, avec le module concerné, le détail de l'action, l'utilisateur qui l'a effectuée et l'horodatage précis. Dans un contexte médical, la traçabilité est une exigence réglementaire. En cas d'audit ou de litige, le journal permet de reconstituer l'historique des opérations et d'identifier l'origine de toute modification. C'est un outil de sécurité et de gouvernance des données.

Q18. Comment l'application gère-t-elle le stock faible des médicaments ?

Chaque enregistrement de stock définit un seuil minimum. La règle de gestion RG-07 stipule qu'une alerte s'affiche lorsque la quantité d'un médicament devient inférieure ou égale au seuil minimum. Cette alerte est visible sur le tableau de bord, ce qui permet au pharmacien de réagir proactivement et de commander un réapprovisionnement avant la rupture de stock. C'est une fonctionnalité essentielle pour éviter les interruptions de traitement.

Q19. Quelles sont les perspectives d'évolution de ce projet ?

Six perspectives principales sont envisagées. Premièrement, l'intégration d'un backend Node.js/Express avec PostgreSQL pour une gestion multi-postes. Deuxièmement, l'authentification JWT pour gérer les sessions utilisateurs. Troisièmement, le développement d'une application mobile avec React Native. Quatrièmement, l'envoi de SMS de rappel de rendez-vous. Cinquièmement, l'intégration de l'intelligence artificielle pour l'aide au diagnostic. Sixièmement, l'interconnexion avec des dispositifs médicaux connectés (IoT) pour la collecte automatique des données vitales.

Q20. Quel a été le plus grand défi technique de ce projet ?

Le plus grand défi a été la conception du framework CRUD générique. Il fallait créer un composant capable de générer automatiquement une interface complète — tableau, formulaires, modales, exports — à partir d'une configuration déclarative, tout en restant suffisamment générique pour couvrir 30 modules aux besoins très différents (patients, médicaments, factures, urgences). Le défi était de trouver le bon niveau d'abstraction : trop générique et l'interface devient rigide ; trop spécifique et on perd l'avantage de l'approche config-driven. La solution a été de définir un schéma de configuration riche avec des types de champs et de colonnes variés, permettant de personnaliser chaque module sans écrire de code.

Conseils pour la soutenance

Gestion du temps : Entraînez-vous avec un chronomètre. 11 minutes passent très vite. Visez 45-50 secondes par diapositive. Si vous prenez du retard, accélérez sur les diapositives techniques et ralentissez sur la problématique et la conclusion.

Posture et voix : Tenez-vous droit, regardez le jury, pas l'écran. Parlez fort, distinctement, à un rythme modéré. Faites des pauses après les points importants pour marquer les transitions.

Gestion des questions : Si une question vous déstabilise, prenez une seconde pour respirer. Commencez par « C'est une excellente question » ou « Si je comprends bien, vous me demandez... » pour vous donner le temps de structurer votre réponse. Si vous ne savez pas, soyez honnête : « Je n'ai pas approfondi cet aspect dans ce projet, mais je pense que la solution serait de... »

Points forts à mettre en avant : L'architecture config-driven (30 modules avec un seul composant), les 26 tests passés à 100%, le design responsive avec mode sombre, et les exports multi-formats. Ce sont vos arguments de différenciation.

À éviter : Ne lisez pas vos diapositives, ne parlez pas trop vite, ne dépassez pas 11 minutes. Ne dites jamais « c'est dans le mémoire » — le jury veut l'entendre de vous, pas lire le mémoire.